

Project Report

Group Identification: Group 9

Gabriel Inumaru Esteves - USP Number: 15453487

Matheus Spineli de Paiva - USP Number: 14598682

Pedro Luís Anghievick - USP Number: 15656521

1 Requirements

The application is a 2D Space Invaders clone that fulfills all assigned requirements: utilizing LibGDX for screen management and graphics, core mechanics (movement, shooting, and collision detection), a local two-player cooperative mode with shared screen and individual lives, a 3-level progression system with increasing difficulty, real-time scoring, and a persistent Save/Load system to persist game progress.

Extra Feature: We introduced a **unique ammunition management system**. Instead of infinite shooting, players must strategically catch random ammo drops to reload their weapons, adding a new layer of resource management to the core gameplay.

2 Project Description

2.1 Brief Description

Developed in Java using the LibGDX framework, players control spaceships at the bottom of the screen to defend against alien formations that move downwards and shoot periodically. The application manages multiple states seamlessly (Main Menu, Gameplay, Pause Menu, Level Complete, and Game Over). It tracks real-time scores and lives, ending the game only when all player lives are lost or aliens breach the defense line.

2.2 OOP Concepts Applied

- **Composition:** The `Swarm` class is composed of a 2D array of `Enemy` objects. It acts as a manager for the entire grid, moving them as a single unit while still allowing individual enemies to maintain their own health and sprite states.
- **Polymorphism:** Demonstrated in the `AmmoDrop` class, which overrides the `draw()` method from the abstract `Entity` class to render its sprite rotated by 90 degrees. Furthermore, collision checks dynamically evaluate the `origin` of a `Bullet` using `instanceof` to distinguish friendly fire from enemy attacks.

3 Comments About the Code

- **Hitbox Synchronization:** To prevent collision bugs, the `Entity` class encapsulates positional data. By using `protected` methods, the bounding box (`Rectangle`) is automatically synced whenever `setX()` or `setY()` are called by the subclasses.

- **Safe Object Removal:** We use boolean flags like `isValid` in `Bullet` and `isCollected` in `AmmoDrop`. Instead of removing objects from memory the moment they collide, we flag them and allow an `Iterator` in the main loop to safely sweep them later.

4 Testing (Plan & Results)

The test plan utilizes the JUnit framework (`GameLogicTest`) to validate core mathematical and physical interactions without requiring the LibGDX graphical context. Tests focus on hitbox synchronization, accurate collision detection between player hitboxes and enemy bullets, and proper instantiation of enemies within a generated `Swarm`. All automated tests passed successfully, confirming that the collision logic robustly ignores inactive entities (e.g., dead enemies and destroyed bullets do not trigger false positive collisions).

5 Build Procedures

1. Ensure you have the latest Java Development Kit (JDK 26+), and make installed.
2. Clone the repository containing the source code and assets.
3. Open a terminal or command prompt in the root directory of the project.
4. Run the application using the Makefile wrapper: `make run`

6 Problems

- **State Persistence:** Saving the game state was a major challenge. We had to ensure that the exact X and Y coordinates, health, and alive status of every individual enemy in the 2D array were saved and accurately restored using LibGDX's `Preferences`.
- **Concurrent Iteration:** Early in development, the game crashed when bullets were removed upon hitting an enemy (`ConcurrentModificationException`). This was solved by migrating from standard `for` loops to `Iterator` implementations.

7 Comments

The inclusion of a local two-player mode added complexity to the collision and Game Over states, but alongside the custom ammo system, it significantly improved the final gameplay experience.